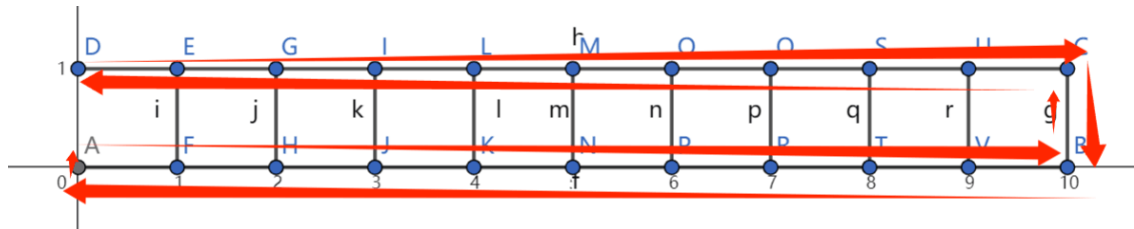


Homework4

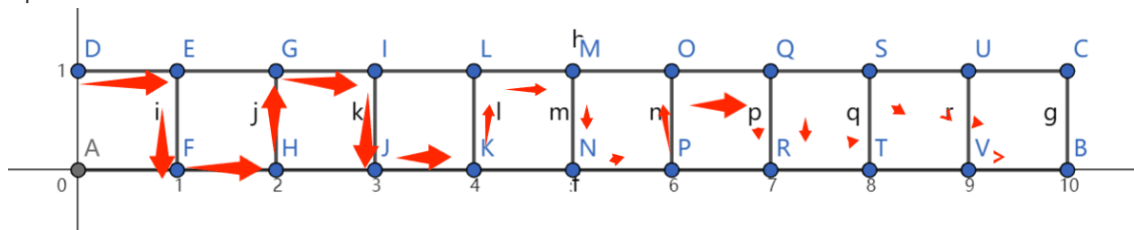
T1

1. Use of N cells:

Greedy tour length: $4n+8$



2. optimal:



3. and the greedy solution can't be improved by inversion

T2

When an inversion-based (2-opt) local search reaches a local minimum, it cannot escape because any immediate 2-opt neighborhood move would result in a worse solution. If we generate the *next* initial solution by simply applying a random 2-opt swap, the next local search phase will likely just reverse that move and immediately fall back into the exact same local minimum.

To prevent this, the "kick" must apply a change that the local search cannot easily undo. Here are two effective ideas:

1. The Double-Bridge Move (Standard TSP Approach):

Instead of a 2-edge change, apply a specific 4-edge change called a "double-bridge." This involves slicing the current locally optimal tour into four segments and reconnecting them in a scrambled order (e.g., A-D-C-B instead of A-B-C-D). Because this move disrupts 4 edges simultaneously, it kicks the solution far enough out of the current local minimum's "basin of attraction" that a 2-opt search (which only fixes 2 edges at a time) cannot trivially step backward into the old trap.

2. Ruin and Recreate (Destruction Phase):

Randomly select a small percentage of the cities (e.g., 10%) and entirely remove them from the current locally optimal tour. Then, use a greedy insertion heuristic to place those cities back into the tour at their cheapest available positions. This creates a brand new starting topology for the next local search run while preserving the well-optimized segments of the previous tour.

T3

In VND, you systematically explore neighborhoods of increasing size or complexity. For the 0/1 Knapsack problem, a solution can be represented as a binary vector $X = (x_1, x_2, \dots, x_n)$ where $x_i = 1$ if item i is in the knapsack, and 0 otherwise.

A standard way to structure the neighborhoods is by the number of item swaps, ordered from least to most disruptive:

- N_1 (**The 1-Flip Neighborhood**):

This neighborhood consists of all feasible solutions that can be reached by changing exactly **one** item. This means either adding one currently unselected item (changing a 0 to a 1, if capacity allows) or dropping one currently selected item (changing a 1 to a 0).

- N_2 (**The 1-in, 1-out Swap Neighborhood**):

This neighborhood consists of all feasible solutions reached by exchanging exactly **one** item inside the knapsack for **one** item outside the knapsack. This keeps the total number of items the same but explores different weight/value combinations.

- N_3 (**The 2-in, 2-out Swap Neighborhood**):

This neighborhood scales up the complexity by exchanging exactly **two** items currently in the knapsack with **two** items currently outside the knapsack, provided the new total weight does not exceed the capacity.

The VND algorithm would start searching in N_1 . If it finds a better solution, it accepts it and stays in N_1 . If N_1 yields no improvements (a local optimum for N_1), it moves to N_2 . If N_2 finds an improvement, it drops back down to N_1 to fine-tune. If N_2 fails, it moves to N_3 , and so on.